

Lab 3: Decomposing your series

MATH840 | Time Series | Week 3 — graded assignment 2 of 7

AUTHOR
Ihor Miroshnychenko

AFFILIATION
Kyiv School of Economics

 Open the notebook

 Open in Colab

Open In Colab

Click **Copy to Drive** before you start working, otherwise nothing you write is saved.

 Want to see it done first?

[Worked example](#) — the whole assignment, start to finish, on Queensland clothing retailing. That series is drawn from `aus_retail`, which is **not** in the pool your own series comes from, so it demonstrates the method without handing you any of your answers.

The task

You have a series. This week you find out **what is in it**, and then you commit to a claim about it.

Concretely: stabilise its variance if it needs it, decompose it two ways, quantify how strong its trend and seasonality are, and predict which of the four simple methods will be hardest to beat on it. Next week you run those methods and find out whether you were right.

 **This one counts**

8 points, produced and submitted **within the session**. Deadline is the end of the practice slot; there is no homework phase. Work is marked afterwards and returned before the next practice session.

What you are marked on

Section	Criteria	Points
1	Data preparation	1
2	EDA and visualisation	4
3	Implementation	2
4	Code quality and reproducibility	1

Section	Criteria	Points
Total		8

The notebook has one section per row. **Keep those headings.** They are how the work is marked, and a report that reorders or drops them is slower to read and scores worse for it.

Your series

The same one you have used since Week 1 — the same student identifier produces the same dataset. If you have lost your notebook, the assignment function is in the template again.

Warning

Do not switch datasets. This is the same series you used last week: last week you found out which simple method wins on it, this week you explain why.

Suggested pacing (80 minutes)

Minutes	What
0–10	Setup, load your series, repair the calendar
10–30	Time plot, seasonal plot, subseries plot, ACF
30–45	Variance: does it need a transformation? Decide and justify
45–60	Classical decomposition and STL; read the components
60–70	F_T , F_S , seasonally adjusted series
70–80	Your prediction, export, submit

If you are behind at minute 70, **submit what you have**. A valid partial submission scores; a perfect missing one does not.

Section 1 — Data preparation (1 point)

Load your series and put it on a regular time grid.

- Resolve duplicated timestamps if there are any, and say in one sentence what you decided and why.
- Fill the calendar so gaps become visible missing values rather than absent rows.
- State the frequency, the span, and how many complete seasonal cycles you have.

You did all of this in Week 1. It should take ten minutes.

⚠️ ⚠️ Where this goes wrong: the frequency string

- "W" does not mean "weekly". It means *week ending Sunday*. If your timestamps are Mondays, a grid built with "W" lines up with nothing and your series comes back empty. Use the anchored form, "W-MON" — which is what `spec["freq"]` already gives you.
- pandas 3 removed the bare "M", "Q", "Y" and "H" aliases. Use "MS" / "ME", "QS" / "QE", "YS" / "YE" and lowercase "h". A bare "Q" now raises rather than warns.
- "MS" is month **start**, "ME" is month **end**. Pick the one your timestamps already use, or every row misses the grid by a few days.

After building a grid, check that you did not lose everything: `s["y"].notna().sum()` should be close to the number of rows you started with.

[offset aliases](#) · [anchored offsets](#) · [date_range](#) · [reindex](#) · [duplicated](#) · [interpolate](#)

Section 2 — EDA and visualisation (4 points)

This is the heaviest section. Four plots and two decompositions, each with a sentence saying what you see — a plot with no reading attached earns nothing.

Graphics. Time plot, seasonal plot, seasonal subseries plot, ACF. Name the patterns: trend, seasonality, cycles, level shifts, outliers.

⚠️ ⚠️ Where this goes wrong

- `plot_acf` raises once `lags` reaches the length of the series. Keep it well below — `lags=min(40, len(y) // 3)` is a safe default, and for weekly data you want at least `lags=60` to see the annual spike.
- Plot the ACF of the series **with missing values removed**; `plot_acf` will not handle `NaN` for you.

[plot_acf](#)

Variance. Does the size of the seasonal swing grow with the level of the series? If it does, the additive decompositions below are the wrong model for it.

- Estimate a Box-Cox λ (`boxcox_lambda` from `coreforecast`, as in the lecture).
- Plot the transformed series next to the original.
- **Decide** whether to work transformed or untransformed, and justify the decision in a sentence. A defensible "no transformation needed" is worth full marks; an unexplained transformation is not.

[boxcox](#) / [boxcox_lambda](#) · [scipy.stats.boxcox](#) · [FPP 3.1 transformations](#) · [FPP the Pythonic Way, ch. 3](#)

⚠️ If your series contains zeros

`boxcox_lambda(..., method="loglik")` refuses non-positive values outright. `method="guerrero"` accepts them but may return $\lambda = 0$, which is the log transform — undefined at zero. A multiplicative decomposition

refuses them for the same reason.

That is information about your series, not an obstacle. Find out where the zeros come from before deciding what to do about them, and say what you concluded. This is exactly the kind of judgement the point is for.

Decomposition. Two of them, on whichever version of the series you settled on:

- classical decomposition (`seasonal_decompose`);
- STL (`STL`), with a stated choice of `seasonal` and `robust`.

Then compare them. Where do they disagree, and which do you trust more on this series?

⚠️ ⚠️ Where this goes wrong

- `seasonal_decompose` returns `NaN` for the first and last `period / 2` observations — six months at each end of a monthly series. That is the centred moving average, not a bug, and it is one of the differences you are asked to comment on.
- `model="multiplicative"` refuses zeros and negative values outright.
- `STL(...)` needs `seasonal` to be an **odd** integer ≥ 3 , and it must be an `int`, not a float. `robust=True` costs a little accuracy and buys resistance to outliers.
- Both need `period` — the number of observations in one cycle, which is `mine["season"]`, not the number of years.

📖 [seasonal_decompose](#) · [STL](#) · [FPP 3.4 classical decomposition](#) · [FPP 3.6 STL](#)

📖 What “reading the components” means

Not “the trend goes up”. Something a colleague could act on: *the seasonal component is stable until 2008 and shifts afterwards, the remainder is flat except for three spikes which line up with the crisis, the trend flattens in the last two years.*

12/34 Section 3 — Implementation (2 points)

Strength of components. Compute, from the STL fit:

$$F_T = \max \left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)} \right) \quad F_S = \max \left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)} \right)$$

Report both, rounded to two decimals.

📖 [FPP 4.3 — where \$F_T\$ and \$F_S\$ come from](#)

Both come out of a **fitted** STL object: `fit.trend`, `fit.seasonal`, `fit.resid`. If you get a value outside $[0, 1]$, you have almost certainly passed the STL object instead of the result of `.fit()`.

Seasonally adjusted series. Produce and plot it. Say what it is useful for and what it hides.

Your prediction. Of the four simple methods — mean, naive, drift, seasonal naive — which will be **hardest** to beat on your series?

to beat on your series:

Set `PREDICTION` to one of `"mean"`, `"naive"`, `"drift"`, `"snaive"`, and argue for it in three or four sentences using F_T , F_S and what you saw in the decomposition.

! Important

The prediction is marked on the **reasoning**, not on being right. A well-argued wrong prediction scores full marks; “snaive, because it usually wins” scores nothing, even though it is usually true.

Section 4 — Code quality and reproducibility (1 point)

- The notebook runs top to bottom on a fresh kernel without errors.
- No dead cells, no leftover experiments, no `df2_final_v3`.
- Every number in your text comes from a cell, not from your memory of an earlier run.

Submission

Upload three files to the Week 3 activity on Moodle **before the end of the session**:

- `SURNAME_lab03.pdf` — the rendered report;
- `SURNAME_lab03.ipynb` — the source;
- `SURNAME_lab03_summary.csv` — written by the last cell of the template: your identifier, F_T , F_S and your prediction.

The summary file is what makes next week’s check instant, so do not skip it.

Reading for next week

Chapter 5 of [Forecasting: Principles and Practice, the Pythonic Way](#) — the forecaster’s toolbox: simple methods, fitted values and residuals, residual diagnostics, prediction intervals, accuracy evaluation.

Next week you implement those four benchmarks on your own series and find out what your prediction was worth.